PRACTICAL WEEK – 6

SESSION-6

Task-1: Create a client-server application using Named Pipes (FIFOs). The client sends messages to the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

```
madhulika@LAPTOP-D2M8NKBH:~$ mkdir fifo_project
madhulika@LAPTOP-D2M8NKBH:~$ cd fifo_project
madhulika@LAPTOP-D2M8NKBH:~/fifo_project$ nano server.c
madhulika@LAPTOP-D2M8NKBH:~/fifo_project$ nano client.c
madhulika@LAPTOP-D2M8NKBH:~/fifo_project$ gcc server.c -o server -pthread
server.c: In function ‘handle_client’:
server.c:30:44: warning: ‘%s’ directive writing up to 1023 bytes into a regi
on of size 994 [-Wformat-overflow=]
   30 |             "Server received your message: %s",
      |                                            ^~
server.c:29:5: note: ‘sprintf’ output between 31 and 1054 bytes into a desti
nation of size 1024
   29 |     sprintf(response,
      |     ^~~~~~~~~~~~~~~~~
   30 |             "Server received your message: %s",
      |             ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
   31 |             req->message);
      |             ~~~~~~~~~~~~~
madhulika@LAPTOP-D2M8NKBH:~/fifo_project$ gcc client.c -o client
madhulika@LAPTOP-D2M8NKBH:~/fifo_project$ ls
client  client.c  server  server.c
madhulika@LAPTOP-D2M8NKBH:~/fifo_project$ cd fifo_project
-bash: cd: fifo_project: No such file or directory
madhulika@LAPTOP-D2M8NKBH:~/fifo_project$ ./server
Server started...
Client 753 sent: Hello server
Response sent to Client 753
Client 846 sent: Hello from Client 2
Response sent to Client 846
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <pthread.h>

#define SERVER_FIFO "/tmp/server_fifo"
#define BUFFER_SIZE 1024

typedef struct {
    int client_pid;
    char message[BUFFER_SIZE];
} Request;

void *handle_client(void *arg)
{
    Request *req = (Request *)arg;

    char client_fifo[100];
    char response[BUFFER_SIZE];

    sprintf(client_fifo, "/tmp/client_%d_fifo", req->client_pid);

    printf("Client %d sent: %s\n",
           req->client_pid, req->message);

    sprintf(response,
            "Server received your message: %s",
            req->message);

    int fd = open(client_fifo, O_WRONLY);

    if (fd == -1) {
        perror("open client FIFO");
        free(req);
```

```
    printf("Response sent to Client %d\n", req->client_pid);

    free(req);

    return NULL;
}

int main()
{
    printf("Server started...\n");

    // Create server FIFO
    mkfifo(SERVER_FIFO, 0666);

    // Open FIFO for reading and writing
    int fd = open(SERVER_FIFO, O_RDWR);

    if (fd == -1) {
        perror("open server FIFO");
        exit(1);
    }

    while (1)
    {
        Request *req = malloc(sizeof(Request));

        if (req == NULL) {
            perror("malloc");
            continue;
        }

        ssize_t bytes = read(fd, req, sizeof(Request));

        if (bytes <= 0) {
            free(req);
            continue;
        }
```

```
  GNU nano 8.7.1                        client.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>

#define SERVER_FIFO "/tmp/server_fifo"
#define BUFFER_SIZE 1024

typedef struct {
    int client_pid;
    char message[BUFFER_SIZE];
} Request;

int main()
{
    int pid = getpid();

    char client_fifo[100];

    sprintf(client_fifo, "/tmp/client_%d_fifo", pid);

    // Create client FIFO
    mkfifo(client_fifo, 0666);

    printf("Client started. PID = %d\n", pid);

    char message[BUFFER_SIZE];

    printf("Enter message: ");
    fgets(message, BUFFER_SIZE, stdin);

    message[strcspn(message, "\n")] = '\0';

    Request req;
```

```
GNU nano 8.7.1                          client.c *
    // Open server FIFO
    int server_fd = open(SERVER_FIFO, O_WRONLY);

    if (server_fd == -1) {
        perror("open server FIFO");
        unlink(client_fifo);
        exit(1);
    }

    // Send request
    write(server_fd, &req, sizeof(Request));

    close(server_fd);

    // Open client FIFO to receive response
    int client_fd = open(client_fifo, O_RDONLY);

    if (client_fd == -1) {
        perror("open client FIFO");
        unlink(client_fifo);
        exit(1);
    }

    char response[BUFFER_SIZE];

    read(client_fd, response, BUFFER_SIZE);

    printf("Server response: %s\n", response);

    close(client_fd);

    // Delete client FIFO
    unlink(client_fifo);

    return 0;
}
```

```
madhulika@LAPTOP-D2M8NKBH:~$ cd ~/fifo_project
madhulika@LAPTOP-D2M8NKBH:~/fifo_project$ ./client
Client started. PID = 753
Enter message: Hello server
Server response: Server received your message: Hello server
madhulika@LAPTOP-D2M8NKBH:~/fifo_project$
```

```
madhulika@LAPTOP-D2M8NKBH:~$ cd ~/fifo_project
madhulika@LAPTOP-D2M8NKBH:~/fifo_project$ ./client
Client started. PID = 846
Enter message: Hello from Client 2
Server response: Server received your message: Hello from Client 2
madhulika@LAPTOP-D2M8NKBH:~/fifo_project$
```

Observation: The client-server application was successfully implemented using Named Pipes (FIFOs). The client was able to send messages to the server, and the server successfully received, processed, and sent responses back to the client. Multiple clients were handled simultaneously using separate threads. Thus, FIFO-based inter-process communication and concurrent client handling were successfully demonstrated.

Task2: Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

```
madhulika@LAPTOP-D2M8NKBH:~/fifo_project$ cd ~
madhulika@LAPTOP-D2M8NKBH:~$ mkdir signal_project
madhulika@LAPTOP-D2M8NKBH:~$ cd signal_project
madhulika@LAPTOP-D2M8NKBH:~/signal_project$ pwd
/home/madhulika/signal_project
madhulika@LAPTOP-D2M8NKBH:~/signal_project$ nano signals.c
madhulika@LAPTOP-D2M8NKBH:~/signal_project$ gcc signals.c -o signals
madhulika@LAPTOP-D2M8NKBH:~/signal_project$ ls
signals  signals.c
madhulika@LAPTOP-D2M8NKBH:~/signal_project$ ./signals
Signal handling program started.
Process ID (PID): 1087

Send signals using:
SIGINT  : Ctrl+C
SIGTERM : kill -SIGTERM 1087
SIGUSR1 : kill -SIGUSR1 1087
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <string.h>

volatile sig_atomic_t sigint_count = 0;
volatile sig_atomic_t sigterm_count = 0;
volatile sig_atomic_t sigusr1_count = 0;

void handle_signal(int signal)
{
    if (signal == SIGINT)
    {
        sigint_count++;

        write(STDOUT_FILENO,
              "\nSIGINT received (Ctrl+C)\n",
              26);
    }

    else if (signal == SIGTERM)
    {
        sigterm_count++;

        write(STDOUT_FILENO,
              "\nSIGTERM received\n",
              19);
    }

    else if (signal == SIGUSR1)
    {
        sigusr1_count++;

        write(STDOUT_FILENO,
              "\nSIGUSR1 received\n",
              19);
```

```c
    sa.sa_handler = handle_signal;

    sigemptyset(&sa.sa_mask);

    sa.sa_flags = 0;

    if (sigaction(SIGINT, &sa, NULL) == -1)
    {
        perror("sigaction SIGINT");
        exit(1);
    }

    if (sigaction(SIGTERM, &sa, NULL) == -1)
    {
        perror("sigaction SIGTERM");
        exit(1);
    }

    if (sigaction(SIGUSR1, &sa, NULL) == -1)
    {
        perror("sigaction SIGUSR1");
        exit(1);
    }

    printf("Signal handling program started.\n");
    printf("Process ID (PID): %d\n", getpid());

    printf("\nSend signals using:\n");
    printf("SIGINT  : Ctrl+C\n");
    printf("SIGTERM : kill -SIGTERM %d\n", getpid());
    printf("SIGUSR1 : kill -SIGUSR1 %d\n\n", getpid());

    while (1)
    {
        printf("Program is running...\n");
        sleep(5);
    }
```

```
madhulika@LAPTOP-D2M8NKBH:~$ kill -SIGUSR1 1087
madhulika@LAPTOP-D2M8NKBH:~$ kill -SIGTERM 1087
madhulika@LAPTOP-D2M8NKBH:~$
```

```
SIGUSR1 received
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...

SIGTERM received
Program is running...
Program is running...
Program is running...
Program is running...
^C
SIGINT received (Ctrl+C)
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
```

Observation: The POSIX signal handling program successfully captured SIGINT, SIGTERM, and SIGUSR1 signals. The signal handler responded asynchronously whenever a signal was received, while the program continued its execution. Thus, asynchronous event handling using POSIX signals was successfully demonstrated.